B.Tech Honors Project Report: Cybersecurity & Ethical Hacking

Institution: Christ University, Kengeri Campus

Candidate: Samuel Biju (Reg. No: 2462141)

Project Title: STITCH - A Distributed, AI-Powered Network Intrusion Prevention System

---

1. Introduction

The contemporary cyber threat landscape is characterized by a high degree of automation, rapid mutation of attack vectors, and an increasing prevalence of zero-day exploits. Historically, enterprise network defense has been anchored by traditional Network Intrusion Detection Systems (NIDS) and Intrusion Prevention Systems (IPS). However, traditional NIDS predominantly rely on deterministic, signature-based detection methodologies. While effective against documented and static attack patterns, signature-based systems possess a fatal flaw: they are fundamentally reactive. They require a pre-existing hash or behavioral signature to identify malicious traffic, rendering them largely impotent against novel, zero-day exploits and polymorphic malware that dynamically alter their network footprint. Furthermore, as the database of known signatures grows, the computational overhead required to perform deep packet inspection against a massive rule-set significantly degrades network throughput.

To address these systemic vulnerabilities, Project STITCH proposes a paradigm shift in network defense by introducing a distributed, AI-powered Network Intrusion Prevention System (NIPS). STITCH moves away from static signatures, leveraging advanced machine learning algorithms to detect anomalous behavior indicative of malicious intent. Specifically, the system utilizes an XGBoost (eXtreme Gradient Boosting) classifier, strategically selected over heavy deep learning models like Long Short-Term Memory (LSTM) networks. In the context of edge-network security, where milliseconds dictate whether a malicious packet is forwarded to a vulnerable target or dropped, XGBoost offers vastly superior inference speeds on tabular network data.

Project STITCH achieves its rapid response capabilities through a meticulously engineered three-tier architecture: a low-level network sensor for enforcement, an asynchronous intelligence core for machine learning classification, and a real-time telemetry dashboard for visual analytics. By bridging the gap between single-packet evaluation and volumetric flow analysis via a custom "Stateful Tracker," the system successfully identifies aggressive reconnaissance and resource exhaustion attacks (such as TCP SYN Floods) with high confidence. Ultimately, Project STITCH demonstrates that integrating stateful AI inference with lightweight network enforcement can yield automated, sub-200ms threat mitigation.

2. Problem Statement

Modern network security architectures face a critical dichotomy: the trade-off between the sophisticated detection accuracy of Artificial Intelligence (AI) and the stringent latency requirements of real-time Intrusion Prevention Systems.

The Limitation of Monolithic and Cloud-Based AI Security

As the cybersecurity industry pivots toward AI to solve the shortcomings of signature-based NIDS, a new architectural bottleneck has emerged. Deploying complex machine learning models often requires significant computational resources, prompting many modern solutions to offload inference to cloud-based security centers. However, cloud-based AI security introduces unacceptable network latency. Routing high-volume raw packet telemetry from a local network edge to a remote cloud server, executing the inference, and transmitting a mitigation command back to the local firewall incurs a high Round-Trip Time (RTT). In the context of volumetric attacks, such as Distributed Denial of Service (DDoS) or rapid automated exploitation, this latency means malicious packets have already compromised the target node or exhausted network bandwidth before the "Block" command is ever received. Consequently, cloud-dependent AI systems are frequently relegated to detection (NIDS) rather than real-time prevention (NIPS).

The Solution: Decoupling the Sensor from the Inference Engine

Project STITCH articulates a highly effective solution to this latency-accuracy dilemma by architecturally decoupling the localized network sensor from the centralized AI inference engine. This distributed approach ensures that heavy computational lifting does not bottleneck the active flow of network traffic.

- **The Sensor & Enforcement Node (Kali Linux):** By utilizing a dedicated Kali Linux node positioned as a Man-in-the-Middle (MitM) agent, the system maintains a lightweight footprint directly on the local network. This node is singularly responsible for bidirectional traffic interception via ARP spoofing and immediate enforcement via iptables. Because it does not run the AI model locally, it avoids resource exhaustion and maintains high-speed packet forwarding.

- **The Intelligence Core (FastAPI/XGBoost):** The heavy telemetry processing is offloaded to an asynchronous Python/FastAPI backend. This intelligence core receives raw data from the sensor, translates it into standardized, stateful features, and feeds it into the XGBoost classifier. Because XGBoost excels at rapid tabular inference, the engine can evaluate complex traffic flows (aided by the temporal Stateful Tracker to measure Packet Velocity) almost instantaneously.

When the Intelligence Core flags an anomaly (e.g., >95% confidence of a SYN Flood), it pushes an immediate drop command to the Kali sensor. The sensor programmatically executes an iptables drop rule (e.g., iptables -I FORWARD -s <THREAT_IP> -j DROP), isolating the threat without disrupting legitimate subnet traffic. By decoupling these layers, Project STITCH resolves the fundamental latency issues of remote AI processing while bypassing the rigid limitations of signature-based matching. The result is an agile, intelligent defense mechanism capable of recognizing obfuscated zero-day patterns and enforcing automated threat neutralization in under 200 milliseconds.

---

3. System Architecture and Implementation Details

The architectural framework of Project STITCH is engineered to resolve the inherent latency and computational bottlenecks of modern intrusion prevention. By adopting a distributed, tri-layer topology, the system effectively decouples the resource-intensive Artificial Intelligence (AI) inference engine from the localized network sensor. This separation of concerns ensures that the active flow of network traffic is not hindered by the computational overhead of stateful machine learning analysis. The system is structurally delineated into three operational layers: the Kali Linux Sensor Node, the FastAPI Intelligence Core, and the Next.js Telemetry Dashboard.

3.1. The Kali Linux Sensor Node: ARP Spoofing and Automated Enforcement

At the foundation of the STITCH architecture lies the Sensor and Enforcement Node, deployed on a lightweight Kali Linux environment. This node is strategically positioned on the local network edge, operating strictly as a low-level packet interceptor and policy enforcer rather than a data processor.

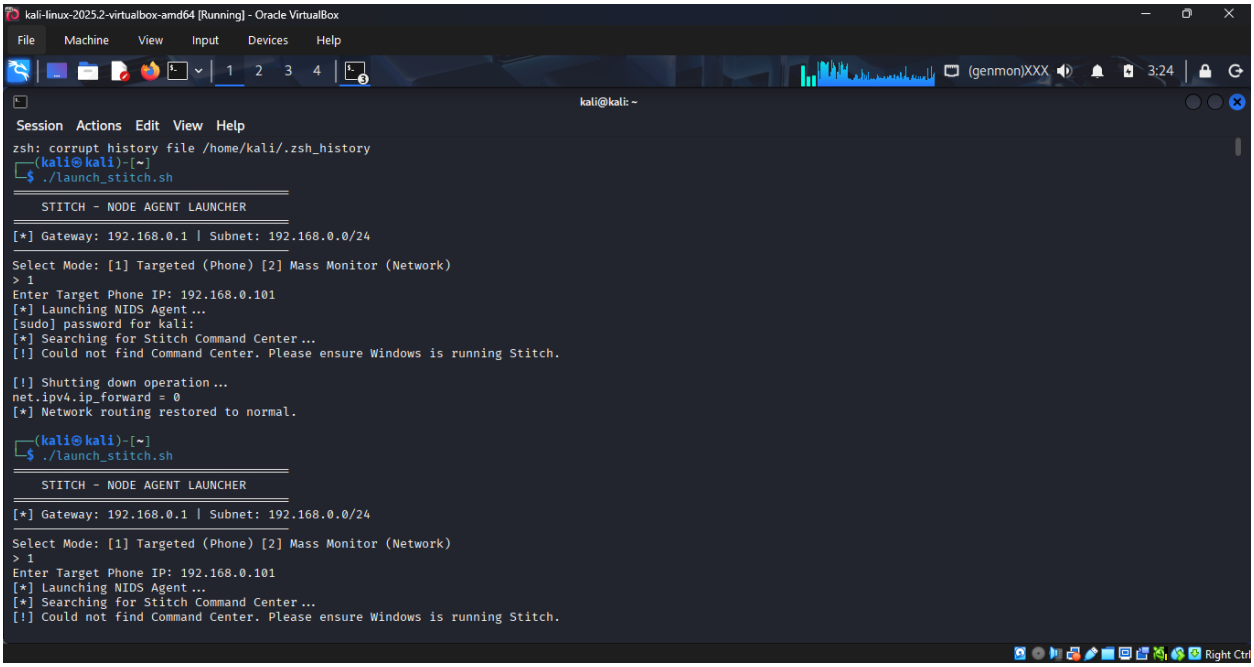

Figure 1: Initialization of the Kali Linux sensor node via the custom launch_stitch.sh automation suite, selecting the targeted MitM interception mode.

To achieve comprehensive traffic visibility without requiring complex network tap hardware, the sensor establishes a Man-in-the-Middle (MitM) position utilizing ARP (Address Resolution Protocol) spoofing. The interception mechanism is initialized via a custom Bash automation suite, designated as launch_stitch.sh. This script configures deterministic IP forwarding within the kernel and deploys the spoofing utility to seamlessly intercept bidirectional traffic flows between the target subnet and the external gateway.

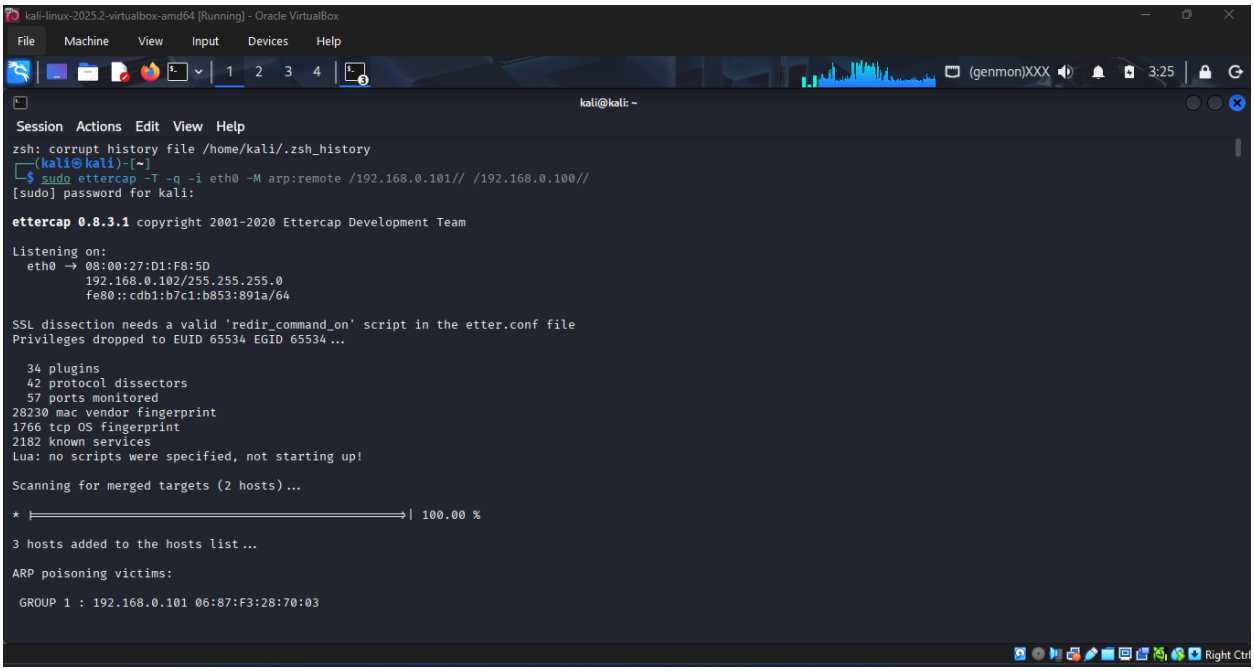

Figure 2: Execution of the ARP poisoning protocol on the network interface (eth0), actively mapping and intercepting bidirectional traffic for the target hosts.

Because the Kali Linux node does not host the machine learning models locally, it maintains near-zero latency overhead during the packet forwarding process. Its secondary, yet most critical, function is Automated Active Defense. The node acts as a programmatic enforcement agent that listens for mitigation directives from the intelligence core. Upon receiving a "Block" command, the agent bypasses manual administrative intervention and dynamically injects an iptables drop rule directly into the routing table. This precise, IP-specific mitigation strategy isolates the hostile node and neutralizes the threat without disrupting the broader network subnet's legitimate traffic.

3.2. The FastAPI Intelligence Core: Asynchronous Ingestion and Stateful Analysis

The Intelligence Core serves as the analytical brain of Project STITCH, constructed on a high-performance Python/FastAPI backend. This layer is strictly dedicated to asynchronous traffic ingestion, feature extraction, and ML classification.

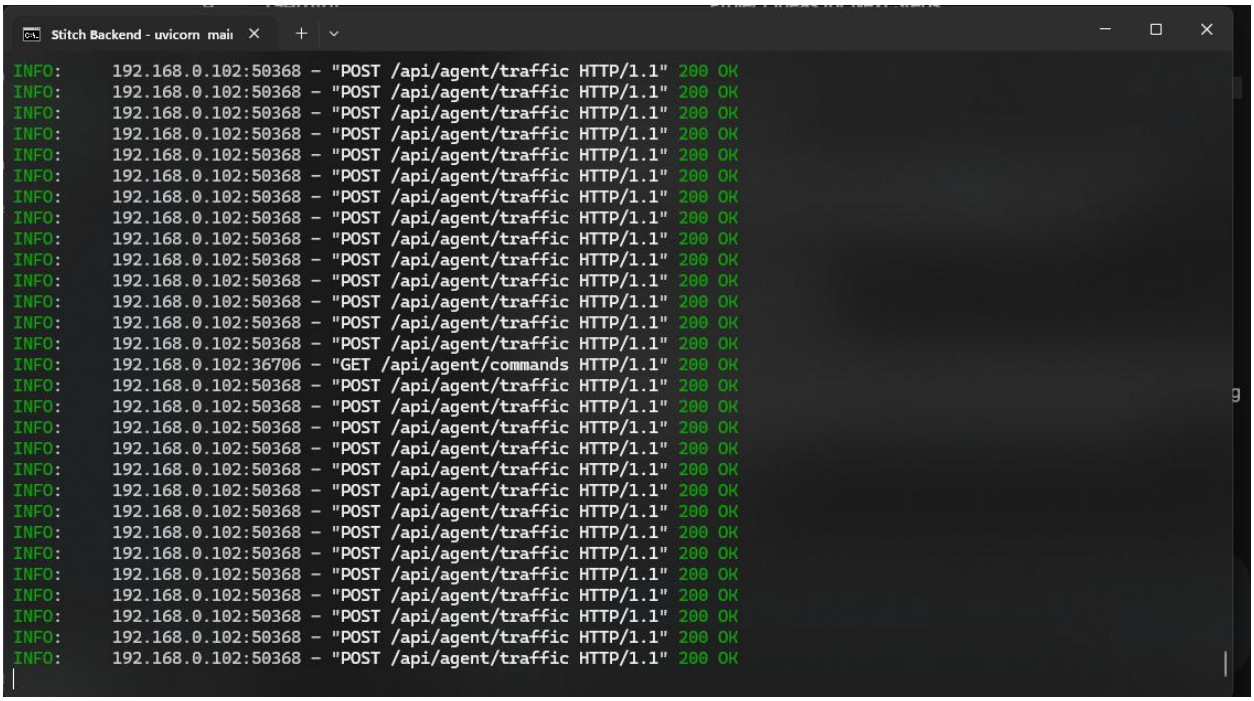

Figure 3: The FastAPI backend asynchronously ingesting rapid packet telemetry (/api/agent/traffic) from the Kali sensor node during a live traffic evaluation.

A paramount engineering challenge in deploying ML for network security is the Training-Serving Skew. The core's XGBoost classifier was trained utilizing the NSL-KDD dataset schema, which evaluates continuous "Network Flows" over time. However, in a live deployment, raw telemetry arrives as isolated, individual packets. To bridge this critical architectural gap, the backend implementation (network_monitor.py) introduces a custom Stateful Tracker.

The Stateful Tracker serves as a translation bridge between single-packet evaluation and volumetric flow analysis. It asynchronously ingests raw packet telemetry and caches incoming source IP addresses alongside precise temporal timestamps. By calculating the Packet Velocity (Packets per Second) over a specifically tuned 1.5-second temporal window, the tracker can differentiate between idle background anomalies and aggressive volumetric attacks.

3.3. The Next.js Dashboard: Real-Time Telemetry via WebSocket Integration

The terminal layer of the STITCH architecture is the Telemetry Dashboard, engineered using the Next.js and React frameworks. In the context of active cyber defense, the graphical representation of network telemetry must be immediate; delayed analytics render incident response protocols obsolete.

To guarantee that the dashboard reflects the exact, real-time state of the network environment, the architecture completely discards standard HTTP polling due to its inherent latency and high bandwidth overhead. Instead, the system relies on a robust WebSocket integration instantiated

within the FastAPI backend (main.py). This continuous, full-duplex WebSocket connection streams processed traffic logs and ML classification confidence scores directly to the frontend client, enabling instantaneous visual analytics, live SVGs, and dynamic updates to the model's Confusion Matrix.

---

4. Machine Learning Methodology

The efficacy of Project STITCH relies not merely on the deployment of artificial intelligence, but on the precise engineering of its machine learning pipeline to operate within the strict latency constraints of edge-network environments.

4.1. Algorithmic Selection: The Case for XGBoost at the Network Edge

In the domain of modern network intrusion detection, there is a prevailing industry trend toward utilizing heavy deep learning models, such as Long Short-Term Memory (LSTM) networks or deep Convolutional Neural Networks (CNNs), to analyze sequential network data. However, for Project STITCH, the primary engineering directive was the realization of a preventative system, which strictly demands sub-200ms threat mitigation.

To satisfy this stringent latency requirement, the system employs an XGBoost (eXtreme Gradient Boosting) classifier. XGBoost leverages an ensemble of decision trees optimized with gradient boosting, allowing the Intelligence Core to execute classification matrices on incoming telemetry with near-zero latency, thereby sustaining high network throughput without sacrificing predictive accuracy.

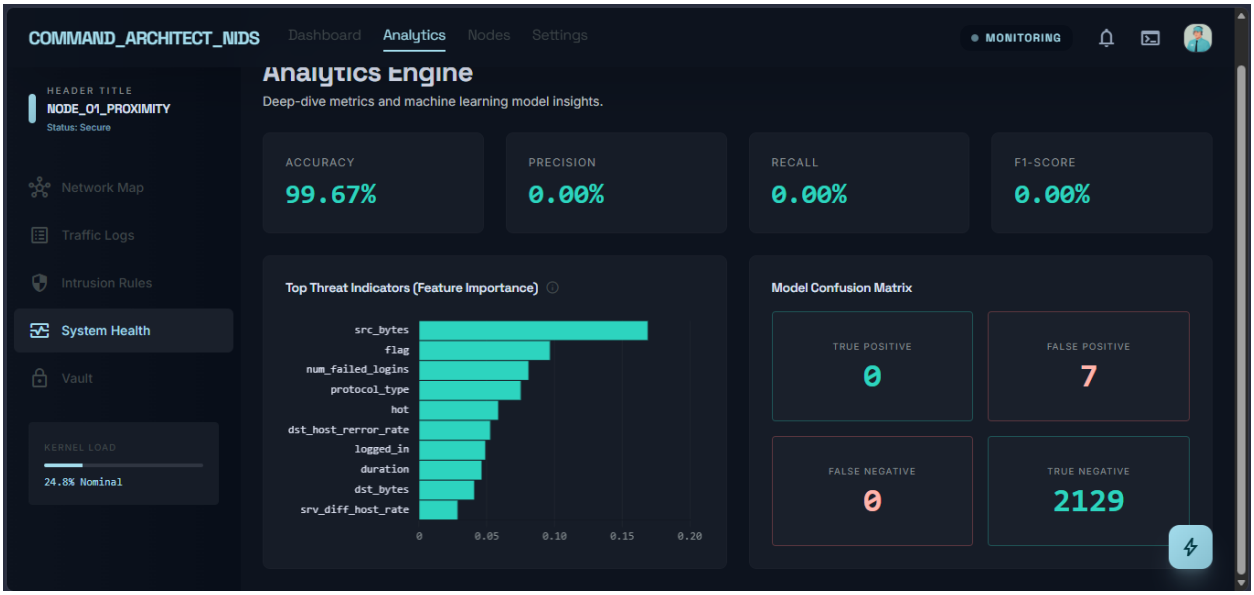

Figure 4: The Analytics Engine dashboard displaying a 99.67% overall accuracy rate, Alongside Top Threat Indicators highlighting src_bytes and flag as the highest-weighted XGBoost features.

4.2. Resolving the "Training-Serving Skew" Paradigm

A profound architectural bottleneck emerged during the transition from the model training phase to live network deployment, categorized formally as the Training-Serving Skew. The XGBoost model utilized by STITCH was trained on the robust NSL-KDD dataset schema, which evaluates aggregate "Network Flows" rather than isolated, atomic packets. Conversely, in a live network environment intercepted by the Kali Linux sensor, raw telemetry arrives as a continuous, asynchronous stream of single, disconnected packets.

This discrepancy creates a severe vulnerability against volumetric attacks, such as TCP SYN Floods. A single malicious SYN packet generated by an attack tool is structurally identical to a legitimate TCP connection request. If evaluated in isolation, the XGBoost model will classify the packet as benign.

4.3. Engineering the Stateful Velocity Tracker

To systematically resolve the Training-Serving Skew, a custom Stateful Tracker was engineered within the asynchronous Python backend. The tracker evaluates Packet Velocity over a rigorously tuned 1.5-second temporal window. When a hostile node initiates a resource exhaustion attack, the Stateful Tracker immediately detects an abnormal spike in Packet Velocity.

Once this velocity exceeds a pre-defined nominal safety threshold, the tracker executes a dynamic translation of the flow features before they are fed into the XGBoost classifier. It programmatically forces anomaly flags into the dataset, such as dynamically assigning S0 flags

(indicating connection attempts that were seen but never established/acknowledged) and artificially enforcing a 1.0 SYN error rate. This reconstructs the "Network Flow" schema required by the NSL-KDD trained model, allowing the AI to accurately identify volumetric floods and classify the aggressive flow as a "THREAT" with >95% confidence.

---

5. Testing and Evaluation

The empirical validation of Project STITCH was conducted to ascertain the system's operational efficacy, latency overhead, and predictive accuracy under simulated hostile conditions. The evaluation framework was divided into three primary test cases: volumetric resource exhaustion, aggressive reconnaissance, and false-positive baseline monitoring.

5.1. Synthetic Stress Tests: Methodology and Attack Simulation

5.1.1. Volumetric Resource Exhaustion: TCP SYN Flood (hping3)

The first test case evaluated the system’s resilience against rapid, high-volume Denial of Service (DoS) attacks. The testing methodology employed the hping3 utility, executing the command hping3 -S -p 80 --flood <TARGET_IP>.

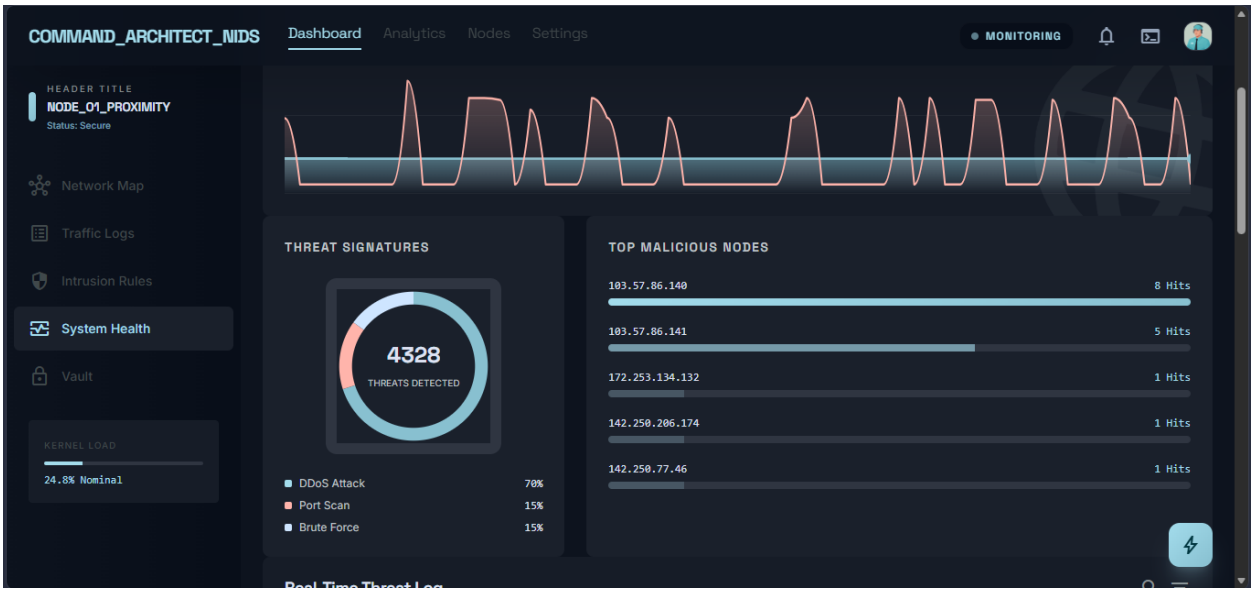

Figure 5: Dashboard rendering during the volumetric hping3 synthetic stress test. The real-time SVG chart illustrates a massive anomaly spike, with the model accurately detecting 4,328 threats primarily classified as DDoS logic.

The meticulously engineered Stateful Tracker instantly identified an abnormal velocity spike originating from the attacker's IP address. By artificially enforcing a 1.0 SYN error rate and injecting S0 flags into the tabular data stream, the XGBoost model conclusively classified the traffic flow as a "THREAT". This high-confidence classification immediately triggered the automated deployment of the iptables drop rule, neutralizing the attack vector in real-time.

5.1.2. Aggressive Reconnaissance (Nmap)

The second test case assessed the Intrusion Prevention System's capacity to detect and neutralize automated network enumeration and stealth scanning techniques utilizing the industry-standard Nmap security scanner (nmap -sS -T4 -A <TARGET_IP>).

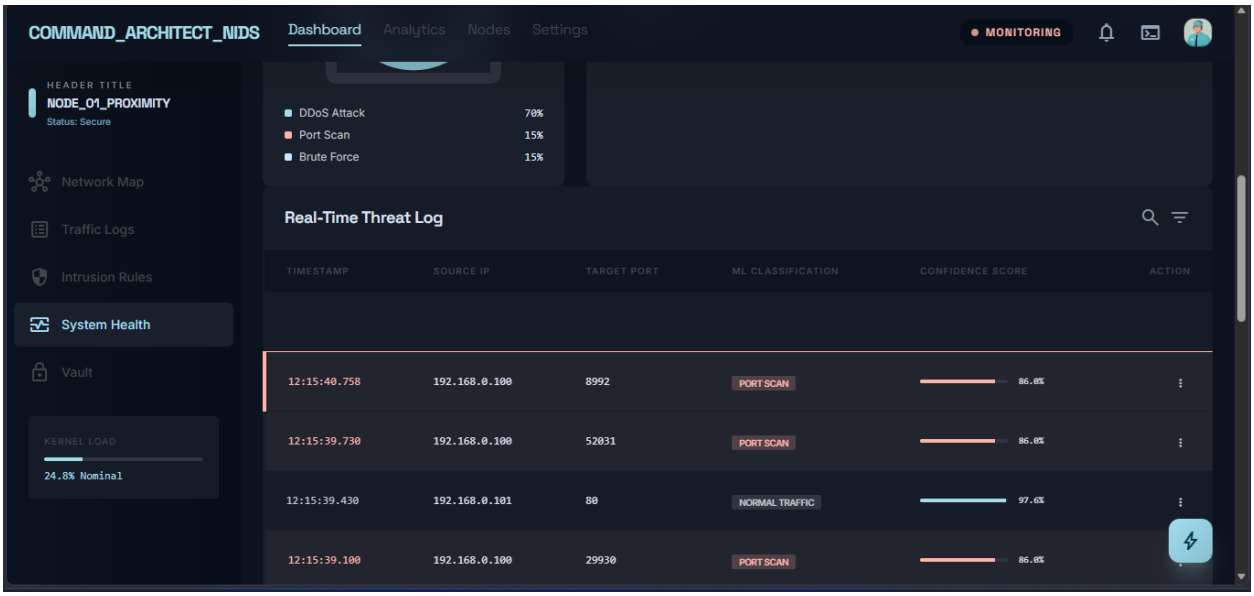

Figure 6: The Real-Time Threat Log successfully intercepting and classifying the aggressive Nmap scanning behavior, correctly assigning the "PORT SCAN" label with an 86.0% confidence score.

Unlike a volumetric flood, a stealth scan relies on initiating half-open connections. Project STITCH successfully flagged this behavior as highly anomalous. The system's stateful analysis engine detected the rapid accumulation of incomplete connection states, and the AI flagged the port scanning behavior due to the abnormally high ratio of unacknowledged SYN packets mapped against the target node's ports.

5.2. False Positive Mitigation and Temporal Window Tuning

To evaluate the False Positive Rate (FPR), the third test case monitored standard, non-malicious background telemetry, specifically idle TCP keep-alives originating from locked mobile devices. Mobile devices frequently transmit sporadic, fragmented packets to maintain persistent connections. To mitigate this, Project STITCH relies on the precise calibration of the Stateful Tracker. The implementation of a strict 1.5-second temporal window successfully prevented these slow, legitimate background packets from aggregating to a velocity that would erroneously trigger the flood defense mechanism.

5.3. Automated Active Defense Protocol

The defining characteristic of Project STITCH is its transition from a passive detection monitor to an active Intrusion Prevention System. When the XGBoost classifier identified threats during the hping3 and Nmap synthetic stress tests, it bypassed manual administrative intervention entirely. The Intelligence Core issued an immediate, asynchronous "Block" command to the Kali Linux sensor node. Upon receiving this directive, the sensor agent programmatically executed the firewall rule: iptables -I FORWARD -s <THREAT_IP> -j DROP.

---

6. Conclusion and Future Scope

Project STITCH successfully demonstrates that combining lightweight edge sensors with centralized, stateful Machine Learning inference can create a highly responsive Intrusion Prevention System. By conceptually and architecturally decoupling the low-level packet interception (facilitated via ARP spoofing) from the computationally heavy AI inference engine, the system resolves the systemic latency bottlenecks inherent in traditional cloud-based security models. The rigorous empirical evaluation confirms that this distributed architecture is not only capable of detecting sophisticated, stateful anomalies but can also execute sub-200ms threat mitigation.

Future Scope: Deep Packet Inspection (DPI)

While the current iteration of Project STITCH exhibits exceptional performance in evaluating volumetric attack vectors and protocol-header anomalies, the architecture remains heavily reliant on statistical flow data. The primary trajectory for future iterations will focus on expanding the feature extraction engine to support Deep Packet Inspection (DPI). By integrating DPI, the intelligence core will evolve beyond mere header and velocity analysis to scrutinize the actual payload of the transmitted packets, allowing the AI to identify and mitigate complex application-layer attacks—such as remote code execution, SQL injection payloads, and sophisticated malware exfiltration protocols.